SCoPe: Trust-Region Latent Planning with JEPA World Models

[TEAM MEMBER 1], [TEAM MEMBER 2]
[Department], *[Institution]*
Guide: [GUIDE NAME]
{[email]}

***Abstract*—Planners built on learned world models choose the action sequence whose *imagined* outcome looks best, but they trust every imagined future equally, including the ones the model gets badly wrong. We study SCoPe, a decision-time check for latent model-predictive control (MPC) with a frozen joint-embedding predictive (JEPA) world model: every imagined step of every candidate plan receives a trust score, and candidates that leave the region where the model is reliable are rejected before a plan is chosen. Building on DINO-WM, we first reproduce its planning results on an 8 GB laptop GPU, made feasible by runtime patches that cut rollout time 3.8× without changing any decision of the cross-entropy-method (CEM) planner: Push-T reaches 0.900 over 3 seeds (published 0.90), Wall 0.947 over 3 seeds (0.96) and PointMaze 1.000 (0.98). We then test the first hypothesis of SCoPe offline: on 76,800 imagined steps checked against the simulator, an untrained Monte-Carlo-dropout score identifies the steps with the largest real latent error at AUROC 0.86 (95% CI 0.84–0.88), above both a random score (0.50) and a strong trivial control (0.79). An offline replay of the gate on recorded candidates, however, barely changes plan success, even with an oracle gate, so the benefit of gating has to be established in closed loop. We report both findings, the protocol, and the remaining steps.**

***Index Terms*—world models, model-based reinforcement learning, model-predictive control, JEPA, uncertainty estimation**

I. Introduction

A robot can plan by imagining the consequences of its actions with a learned world model and executing the action sequence whose imagined result is closest to its goal. Joint-embedding predictive architectures (JEPA) [4], [5] make this practical from images: instead of predicting pixels, they predict the future in the embedding space of a pretrained encoder. DINO-WM [1] combines a frozen DINOv2 encoder [3] with a learned latent predictor and plans with the cross-entropy method (CEM) inside MPC, reaching 90% success on the Push-T manipulation task without any reward or task-specific training; V-JEPA 2-AC [2] applies the same idea to real robot arms.

These planners have a blind spot: the planner selects the candidate whose imagined future scores best and never asks whether that imagination is reliable. Fig. 1 shows a case from our own runs: the model imagines the T-shaped block rotating onto the goal, the planner commits to the plan, and in the simulator the block barely moves. Prediction errors also grow with the horizon, and simply penalising uncertainty can backfire [18].

Research question. Can an uncertainty score computed on the imagined latent states flag unreliable candidate plans well enough that rejecting them improves real success, without discarding the good plans?

Contributions. (i) A reproduction of DINO-WM planning on Push-T, Wall and PointMaze on an 8 GB GPU, with runtime patches that are verified not to change the planner's choices. (ii) A protocol that labels every imagined step of CEM's candidates with its real latent error by executing the candidates in the simulator. (iii) An offline test (H1) of five untrained trust scores against a random and a trivial control, with bootstrap intervals over decision points. (iv) The design of the SCoPe gate, a fallback ladder and the closed-loop evaluation plan with random and oracle gates. The closed-loop evaluation itself is future work, and we say so wherever it matters.

II. Background and Related Work

Uncertainty in model-based RL. PETS [6] plans with an ensemble of probabilistic models; MBPO [8] limits model rollouts to short horizons; MOPO [9] and MOReL [10] penalise or halt rollouts in uncertain regions for offline RL; MACURA [11] adapts the rollout length to model uncertainty. All of them distrust the model *while training a policy*, mostly on low-dimensional states.

Latent world models. PlaNet [7], DreamerV3 [12] and TD-MPC2 [13] learn latent dynamics with a reward; JEPA-style planners such as DINO-WM [1], PLDM [14], HWM [15] and V-JEPA 2-AC [2] are reward-free and plan towards a goal image, but they trust every imagined rollout equally. Latent safety filters [16] reject actions in latent space for safety constraints rather than prediction errors.

Warnings. Ensemble disagreement can miss compounding error in latent models [17]; uncertainty penalties raised collisions in a navigation study [18]; and diagnostics of JEPA planners point to action-consistency [19], representation geometry [20] and stage-wise reliability [21]. We therefore treat every trust score as a hypothesis to be tested against real error, with deep ensembles [22] and MC-dropout [23] as the classical candidates.

Our difference. SCoPe changes one step of the planning loop: it scores each imagined step of each CEM candidate at decision time, with a frozen backbone, and rejects untrusted candidates before a plan is chosen.

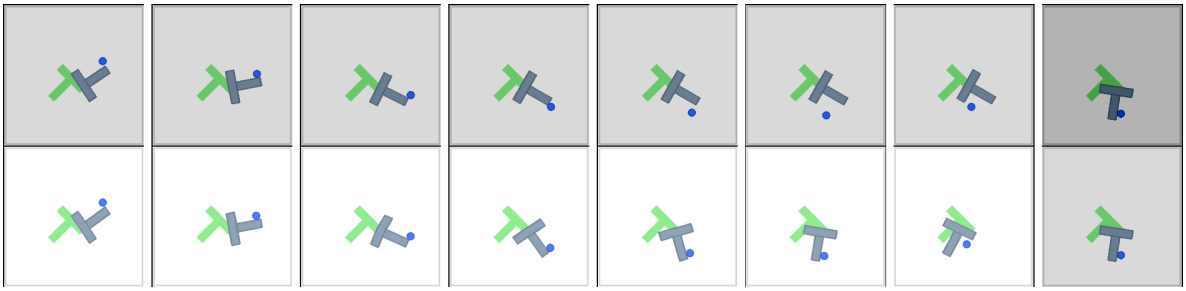


Fig. 1. A logged failure (Push-T, seed 101, episode 0, simulator steps 0–25 and the goal). Top: what happened in the simulator. Bottom: what DINO-WM imagined for the same actions. The imagined block reaches the goal pose; the real one barely moves, and the episode is never solved. SCoPe aims to flag such imagined steps before the plan is chosen.

III. Problem Formulation

We model goal-reaching as a finite-horizon MDP without a hand-designed reward. The observation o_t is a 224×224 image and the agent's proprioceptive state p_t. The frozen encoder maps it to the latent state $z_t = E(o_t) \in \mathbb{R}^{196\times384}$ (DINOv2 ViT-S/14 patch tokens). An action is a 2-D target displacement; actions are grouped in chunks of five simulator steps, $a_k \in \mathbb{R}^{10}$. The predictor f_θ imagines $\hat{z}_{k+1} = f_\theta(\hat{z}_{k-2:k}, a_k)$ from a three-frame history. Real transitions come from the simulator. An episode succeeds when the environment's own criterion holds at the end (for Push-T, block and agent within 20 units and $\pi/9$ of the goal pose).

Given the goal latent z_g, DINO-WM plans $H = 5$ chunks ahead by minimising $C(a_{1:H}) = \|\hat{z}_H - z_g\|^2 + \alpha\,\|\hat{p}_H - p_g\|^2$. SCoPe adds a constraint, not a reward:

$$\min_{a_{1:H}} C(a_{1:H}) \quad \text{s.t.} \quad u(\hat{z}_k) \le \tau \;\text{ for all } k = 1,\dots,H, \qquad (1)$$

where u is a trust score (higher means less trusted) and τ is set on held-out decision points. This is a sequential decision problem rather than a supervised one because a push takes effect several steps later and success is judged at the end; we use CEM-MPC rather than PPO or SAC because the data are offline and reward-free, and because CEM keeps an explicit set of candidates that a gate can filter.

IV. Method: SCoPe

A. Architecture

Fig. 2 shows the system. At each decision the simulator returns an observation, the planner asks the world model to encode it and imagine the candidates, the trust module scores every imagined step, and only accepted candidates compete on cost. Offline, imagined latents are logged next to the encodings of what really happened; this error-labelled data calibrates the scores and τ. The world model is never retrained. Backbones and scorers plug in through two small interfaces, so V-JEPA 2-AC can replace DINO-WM later.

B. Trust scores

All scores are computed from the imagination alone, need no training, and are larger for less trusted steps:

- **MC-dropout** [23]: variance of $\hat{z}_k$ over four predictor passes with its dropout (0.1, as trained) on.
- **Action perturbation**: divergence of the imagined rollout when the actions receive Gaussian noise ($\sigma = 0.1$).
- **Image perturbation**, in the spirit of [19]: divergence when the start image receives noise ($\sigma = 0.1$).
- **Distance to training data**: mean distance to the five nearest encoded training frames (random projection to 512 dimensions).
- **Imagined cost**: the plan's own predicted distance to the goal.
- Controls: **distance moved**, $\|\hat{z}_k - z_0\|$, a trivial score, and **random**.

Algorithm 1 SCoPe decision step (one MPC round)

observe o_t; $z_t \leftarrow E(o_t)$
for CEM iteration $i = 1,\dots,I$ **do**
  sample N action sequences $a^{(n)}_{1:H} \sim \mathcal{N}(\mu, \Sigma)$
  imagine $\hat{z}^{(n)}_{1:H}$ with f_θ; score $u^{(n)}_k$ for every step
  accepted $\mathcal{A} \leftarrow \{n : \max_k u^{(n)}_k \le \tau\}$
  if $|\mathcal{A}| < K$ **then**
    fallback: resample wider $\rightarrow$ $H \leftarrow H/2$ $\rightarrow$ no-op (logged)
  end if
  refit (μ, Σ) to the K lowest-cost candidates in $\mathcal{A}$
end for
execute the first chunks of the best accepted plan; return to line 1

C. Gated CEM-MPC with a fallback ladder

Algorithm 1 gives the decision loop. Imagined latents are never used as the start of the next plan: after executing the accepted actions, the agent observes again. When too few candidates pass, the ladder widens the search, then shortens the horizon, then takes a safe no-op, and logs which rung fired.

D. Hypotheses

H1: a trust score ranks steps with large real latent error above the others (AUROC, Spearman ρ). H2/H3: the gate's false rejections (plans that would have succeeded) and false acceptances stay below a random gate's. H4: gating raises real success over plain MPC, a soft uncertainty penalty and a random gate, with an oracle gate (rejecting on the true error)

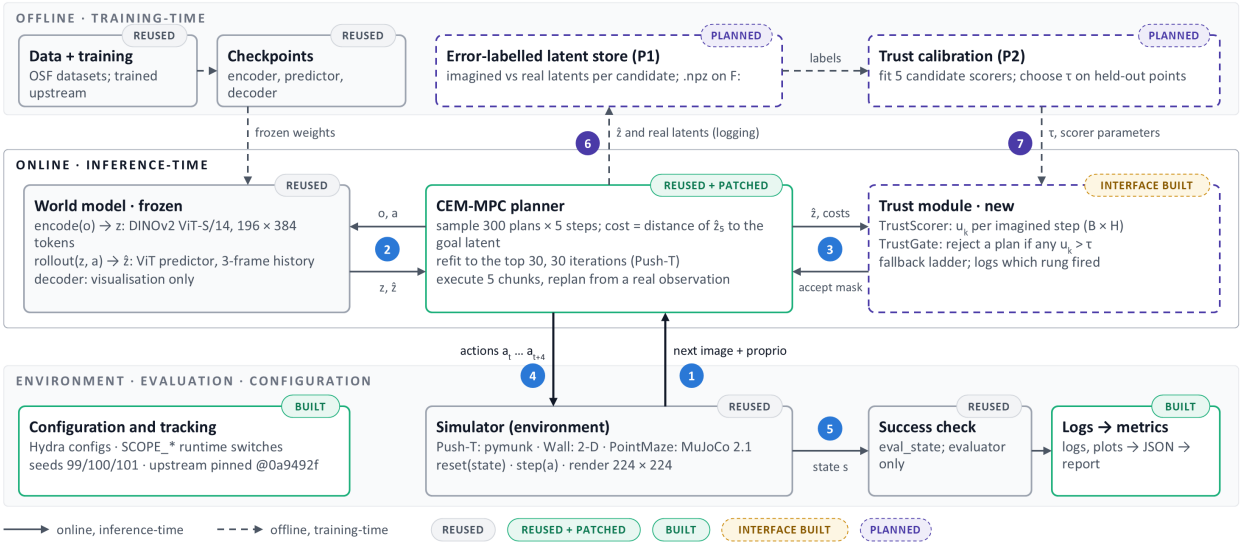


Fig. 2. SCoPe system architecture. Middle: the agent at decision time (frozen DINO-WM, CEM-MPC planner and the trust module). Top: the offline loop that logs imagined against real latents and calibrates the scores. Status tags mark reused, built, interface-only and planned components.

as the upper bound. Go/no-go: the closed loop is built only if a score beats both the random and the trivial control on H1 with non-overlapping 95% intervals.

V. Experimental Setup

Tasks and data. Push-T [24] (push a T-shaped block onto a target), Wall (pass through a door) and PointMaze [25], with the datasets and released checkpoints of DINO-WM [26]. Push-T goals are the real state 25 steps later on a held-out trajectory; Wall and PointMaze goals are sampled reachable states.

Planner. Upstream settings: 300 samples, top 30, horizon 5, 30 CEM iterations on Push-T and 10 on Wall and PointMaze, five chunks executed per MPC round, at most three rounds (the paper does not state its replanning budget, so we report every round). 50 start/goal pairs per seed; seeds 99, 100 and 101. We read the paper's CEM row as our round 1 (one plan, executed open loop) and its MPC row as success after replanning.

Hardware and runtime patches. One RTX 5060 laptop GPU (8 GB), WSL2, PyTorch 2.7.1 with CUDA 12.8. Unmodified, a 300-candidate rollout took 26.2 s and did not fit in memory at full settings. We fused attention, encoded the shared observation once instead of 300 times, chunked rollouts, disabled autograd where unused, and enabled TF32 matrix multiplication: 6.9 s per rollout (3.8×). Against the unmodified model, CEM's top-30 set and its update were identical at three sampling spreads; bf16 was faster but changed 17% of the elites at small spreads and was rejected. (These benchmark figures are from our benchmark script, not from the generated tables.)

H1 pilot protocol. On 80 Push-T decision points over 3 seeds, CEM ran 10 iterations; we recorded 192 of the candidates it evaluated per decision point (48 from iteration 1, 48 from iteration 5 and 96 from iteration 10, so the set spans bad to good plans), imagined each, executed each from the same start state in the simulator, and encoded the real frames at steps $5, 10, \ldots, 25$: 76,800 imagined steps with a real latent next to them. The real error of step k is the mean squared difference of the imagined and real patch tokens; a step is *unreliable* if its error is in the top 10% *for its step index*, so that the growth of error with the horizon cannot inflate a score. AUROC and Spearman ρ are computed per step and averaged; 95% intervals come from 500 bootstrap resamples over decision points.

TABLE I
Planning success over 50 start/goal pairs per seed (mean ± SD over seeds).

Task	Seeds	One plan (round 1)	With replanning	Paper [1]
Push-T	3	0.87 ± 0.05	0.90 ± 0.05 (round 3)	0.86 / 0.90
Wall	3	0.52 ± 0.02	0.95 ± 0.02 (round 3)	0.74 / 0.96
PointMaze	1	0.86	1.00 (round 2)	0.80 / 0.98

Offline gate replay. Among each decision point's recorded candidates, plain selection takes the lowest imagined cost; a gate first rejects the 30% least trusted candidates (by their worst step). The outcome is whether the chosen candidate succeeds in the simulator. This is a hint for H4, not H4.

VI. Results

A. Reproduction of DINO-WM

Table I summarises the reproduction. On Push-T we obtain 0.873 ± 0.046 after one plan and 0.900 ± 0.053 after replanning (per seed 0.92 / 0.94 / 0.84), against the published 0.86 and 0.90; at 50 pairs the binomial standard error is about 0.04, so both published values lie within the seed spread. PointMaze reaches 1.000 after round 2 (published 0.98). Wall

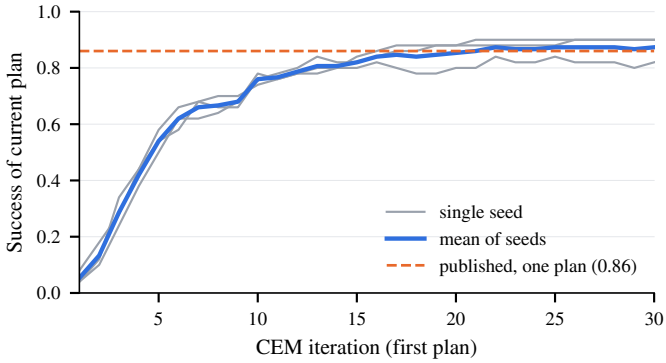


Fig. 3. Push-T, first plan: success rate of CEM's current mean plan after each iteration (evaluation only; the planner never sees it). Grey: seeds; blue: mean; dashed: published single-plan result.

TABLE II
H1 PILOT: AUROC FOR THE TOP-10% REAL LATENT ERROR (PER STEP), 80 PUSH-T DECISION POINTS, 3 SEEDS.

Trust score	AUROC	95% CI	Spearman ρ
MC-dropout (4 passes)	**0.863**	**0.84–0.88**	0.65
Action perturbation	0.657	0.64–0.67	0.27
Distance to training data	0.619	0.56–0.66	0.12
Imagined cost	0.511	0.47–0.55	0.04
Image perturbation	0.496	0.44–0.55	−0.17
MC-dropout + distance (rank avg.)	0.851	0.83–0.87	0.64
Distance moved (control)	0.790	0.76–0.82	0.51
Random (control)	0.500	0.49–0.51	0.00

reaches 0.947 after round 3 (per seed 0.92 / 0.96 / 0.96; published 0.96), but its single-plan result, 0.520 ± 0.020, is consistently below the published 0.74. We do not know why yet; the upstream Wall setting stops CEM after 10 iterations, and our mapping of the paper's CEM row to round 1 may not match its protocol. Fig. 3 shows that on Push-T the success of CEM's current plan rises steeply in the first ten iterations and plateaus after about twenty.

B. H1: do trust scores predict real error?

Table II and Fig. 4 answer H1 for this pilot. MC-dropout ranks the unreliable steps highest, with AUROC 0.86 (95% CI 0.84–0.88; per seed 0.87, 0.84, 0.88), and its whole interval lies above the trivial control (0.79, 0.76–0.82) and the random score (0.50). It is the only score that passes the go/no-go criterion. Action perturbation (0.66) and distance to the training data (0.62) carry some signal but less than the trivial control. The plan's own imagined cost carries none (0.51): a confident planner is no evidence of a correct imagination. Image perturbation is at chance (0.50), most likely because small noise on the input image is absorbed by the frozen encoder, so it probes the encoder rather than the predictor. The strength of the trivial control is itself informative: plans that change the scene a lot are the ones the model gets wrong, so every score must be compared with it, not only with chance. A rank average of MC-dropout and the trivial control, fixed in advance, does not improve on MC-dropout alone (Table II).

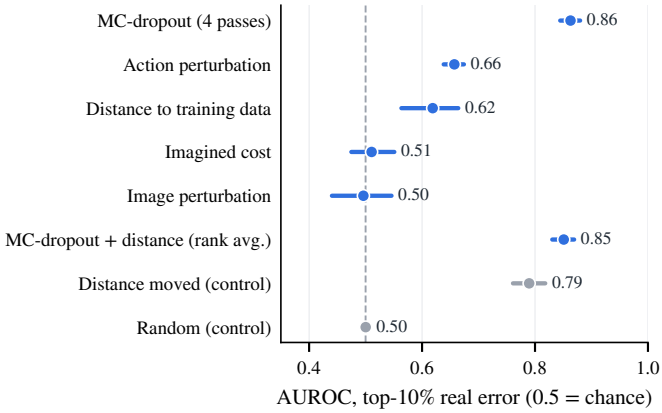


Fig. 4. AUROC with 95% bootstrap intervals over decision points. Blue: candidate trust scores; grey: controls; dashed: chance.

TABLE III
OFFLINE GATE REPLAY (REJECT 30%, THEN LOWEST IMAGINED COST).

Selection among recorded candidates	Chosen plan succeeds
Plain (lowest imagined cost)	71.3%
Oracle gate (true error)	72.5%
Random gate	70%
MC-dropout gate	70%
Action-perturbation gate	75%
Distance-to-data gate	68.8%
Distance-moved gate	67.5%

C. Offline gate replay

Table III shows the replay. The plan chosen without a gate succeeds in 71.3% of decision points, and 70% with the MC-dropout gate; one decision point is worth 1.3%, so the differences between gates are within noise. The informative row is the oracle: even rejecting by the *true* error barely changes success (72.5% against 71.3%). Among the recorded candidates, the one with the lowest imagined cost is rarely a high-error one, so a one-shot gate has little to change. Predicting error (H1) is therefore necessary but not sufficient; whether rejection changes what the agent does over several replanning rounds, and in harder conditions, is the question for H4.

VII. DISCUSSION AND LIMITATIONS

The evidence for SCoPe so far is offline and from one task: three seeds and 80 decision points, one sampling context (CEM's first plan), no calibration error yet, and no closed-loop result. The pilot shows that the logging pipeline works and that a cheap, untrained score tracks real error well beyond a strong trivial baseline; it does not show that gating helps, and the oracle replay suggests the gain, if any, will come from changing decisions across rounds and from harder or shifted conditions rather than from filtering the final candidates. All tasks are simulated and 2-D; real-world validity would require a backbone such as V-JEPA 2-AC with robot data. Planning takes minutes per decision on our GPU, so the system is not real-time. Upstream MPC stops solved episodes using

the simulator's success flag; we keep this identical for every method.

VIII. Conclusion and Future Work

We reproduced DINO-WM planning on an 8 GB GPU within the published results on Push-T, PointMaze and, after replanning, Wall, with a 3.8× faster runtime that leaves the planner's choices unchanged. A first test of SCoPe's central hypothesis found that an untrained MC-dropout score identifies the most wrong imagined steps (AUROC 0.86), clearly above random and above a strong trivial control, while an offline gate replay showed almost no change in success, even for an oracle. Next steps: more decision points, the remaining sampling contexts (dataset states, perturbed starts and visual shifts that leave the dynamics unchanged), calibration, an ensemble score, and then the closed-loop gate with its fallback ladder against plain MPC, a soft penalty, a random gate and the oracle gate on Push-T, Wall and PointMaze.

Reproducibility

Code, run scripts and the report sources are in the project repositories (scope-planning, scope-project, scope-mid-review). Every number in Tables I–III and in the text is generated from the run logs by `report/make_assets.py`.

References

[1] G. Zhou, H. Pan, Y. LeCun, L. Pinto. DINO-WM: World Models on Pre-trained Visual Features enable Zero-shot Planning. ICML 2025, PMLR 267:79115–79135. https://arxiv.org/abs/2411.04983

[2] M. Assran et al. V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning. arXiv:2506.09985, 2025. https://arxiv.org/abs/2506.09985

[3] M. Oquab et al. DINOv2: Learning Robust Visual Features without Supervision. TMLR, 2024. https://arxiv.org/abs/2304.07193

[4] M. Assran et al. Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture (I-JEPA). ICCV 2023. https://arxiv.org/abs/2301.08243

[5] Y. LeCun. A Path Towards Autonomous Machine Intelligence. Position paper, v0.9.2, 2022 (not peer-reviewed). https://openreview.net/forum?id=BZ5a1r-kVsf

[6] K. Chua, R. Calandra, R. McAllister, S. Levine. Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models (PETS). NeurIPS 2018. https://arxiv.org/abs/1805.12114

[7] D. Hafner et al. Learning Latent Dynamics for Planning from Pixels (PlaNet). ICML 2019, PMLR 97:2555–2565. https://arxiv.org/abs/1811.04551

[8] M. Janner, J. Fu, M. Zhang, S. Levine. When to Trust Your Model: Model-Based Policy Optimization (MBPO). NeurIPS 2019. https://arxiv.org/abs/1906.08253

[9] T. Yu et al. MOPO: Model-based Offline Policy Optimization. NeurIPS 2020. https://arxiv.org/abs/2005.13239

[10] R. Kidambi, A. Rajeswaran, P. Netrapalli, T. Joachims. MOReL: Model-Based Offline Reinforcement Learning. NeurIPS 2020. https://arxiv.org/abs/2005.05951

[11] B. Frauenknecht, A. Eisele, D. Subhasish, F. Solowjow, S. Trimpe. Trust the Model Where It Trusts Itself: Model-Based Actor-Critic with Uncertainty-Aware Rollout Adaption (MACURA). arXiv:2405.19014, 2024. https://arxiv.org/abs/2405.19014

[12] D. Hafner, J. Pasukonis, J. Ba, T. Lillicrap. Mastering Diverse Control Tasks through World Models (DreamerV3). Nature 640:647–653, 2025. https://doi.org/10.1038/s41586-025-08744-2

[13] N. Hansen, H. Su, X. Wang. TD-MPC2: Scalable, Robust World Models for Continuous Control. ICLR 2024. https://arxiv.org/abs/2310.16828

[14] V. Sobal et al. Learning from Reward-Free Offline Data: A Case for Planning with Latent Dynamics Models (PLDM). NeurIPS 2025. https://arxiv.org/abs/2502.14819

[15] W. Zhang et al. Hierarchical Planning with Latent World Models (HWM). arXiv:2604.03208, 2026. https://arxiv.org/abs/2604.03208

[16] J. Seo, K. Nakamura, A. Bajcsy. Uncertainty-aware Latent Safety Filters for Avoiding Out-of-Distribution Failures. CoRL 2025. https://arxiv.org/abs/2505.00779

[17] J. Berger, B. Frauenknecht, S. Trimpe, B. Leibe. Biased Dreams: Limitations to Epistemic Uncertainty Quantification in Latent Dynamics Models. RLC 2026. https://arxiv.org/abs/2604.25416

[18] Z. Wang. Learning from World Feedback: Why Model Uncertainty Fails as a Risk Signal in Model-Based RL. ICML 2026 Workshop on RL from X Feedback. https://arxiv.org/abs/2607.16591

[19] G. An et al. Diagnosing JEPA World Models with Action-Conditioned Predictive Consistency (ACPC). arXiv:2608.12939, 2026. https://arxiv.org/abs/2608.12939

[20] J. Hu et al. Representation Geometry Matters for Planning with JEPA World Models. arXiv:2608.16287, 2026 (arXiv title spells "Reperesentation"). https://arxiv.org/abs/2608.16287

[21] G. Lee, B.-T. Zhang. Beyond Task Success: Stage-Wise Reliability of World Model Planning under Sensing Degradation. arXiv:2609.07126, 2026. https://arxiv.org/abs/2609.07126

[22] B. Lakshminarayanan, A. Pritzel, C. Blundell. Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles. NIPS 2017. https://arxiv.org/abs/1612.01474

[23] Y. Gal, Z. Ghahramani. Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning. ICML 2016. https://arxiv.org/abs/1506.02142

[24] C. Chi et al. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. RSS 2023 (origin of the Push-T task used by DINO-WM). https://arxiv.org/abs/2303.04137

[25] J. Fu, A. Kumar, O. Nachum, G. Tucker, S. Levine. D4RL: Datasets for Deep Data-Driven Reinforcement Learning. arXiv:2004.07219, 2020 (PointMaze). https://arxiv.org/abs/2004.07219

[26] DINO-WM code (MIT licence), pinned at commit 0a9492f; datasets and checkpoints on OSF (project bmw48). https://github.com/gaoyuezhou/dino_wm